SolitityExchange.sol

SolitityExchange.sol

Purpose:

Hosts verified businesses as listings

Accepts revenue proofs (IPFS hash)

Issues SST (Sotility Stable Tokens) backed by real revenue

Emits events for DAO/Oracle tracking

Designed to distribute dividends or revenue shares later

A tokenized business launchpad — fully compatible with the SST stablecoin, IPFS proofing, and DAO governance.

Key Concepts

Businesses are onboarded via registerBusiness(...)

Revenue is submitted via submitRevenueProof(...)

SST is minted to the business wallet or treasury

The DAO or AI verifies IPFS revenue data

Assumptions

Your SST contract must support mint() by this contract (we can add a MINTER_ROLE)

IPFS hashes link to receipts and are verifiable off-chain or by AI oracles

Can plug directly into DAO governance for listed business approval

--

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/access/AccessControl.sol";
import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import "@openzeppelin/contracts/token/ERC20/extensions/ERC20Burnable.sol";

/**
 * @title SotilityExchange
 * @dev Tokenized listing and SST issuance for revenue-verified businesses.
 */
contract SotilityExchange is AccessControl {
    bytes32 public constant BUSINESS_ROLE = keccak256("BUSINESS_ROLE");
    bytes32 public constant VERIFIER_ROLE = keccak256("VERIFIER_ROLE");

    struct Business {
        string name;
        string description;
        string ipfsMetadata;
        address wallet;
        bool active;
    }

    struct RevenueProof {
        string ipfsHash;
        uint256 amount;
        uint256 timestamp;
    }

    mapping(address => Business) public businesses;
    mapping(address => RevenueProof[]) public revenueProofs;

    IERC20 public stableToken; // e.g. SST
    address public treasury;

    event BusinessRegistered(address indexed wallet, string name);
    event RevenueSubmitted(address indexed business, uint256 amount, string ipfsHash);
    event SSTIssued(address indexed to, uint256 amount);

    constructor(address _stableToken, address _treasury) {

_grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
        stableToken = IERC20(_stableToken);
        treasury = _treasury;
    }

    function registerBusiness(
        address wallet,
        string calldata name,
        string calldata description,
        string calldata ipfsMetadata
    ) external onlyRole(DEFAULT_ADMIN_ROLE) {
        require(!businesses[wallet].active, "Already registered");
        businesses[wallet] = Business({
            name: name,
            description: description,
            ipfsMetadata: ipfsMetadata,
            wallet: wallet,
            active: true
        });
        _grantRole(BUSINESS_ROLE, wallet);
        emit BusinessRegistered(wallet, name);
    }

    function submitRevenueProof(address business, uint256 amount, string calldata ipfsHash)
        external onlyRole(VERIFIER_ROLE)
    {

require(businesses[business].active, "Business not registered");
        revenueProofs[business].push(RevenueProof({
            ipfsHash: ipfsHash,
            amount: amount,
            timestamp: block.timestamp
        }));

        ERC20Burnable(address(stableToken)).mint(business, amount); // Assume minting rights exist
        emit RevenueSubmitted(business, amount, ipfsHash);
        emit SSTIssued(business, amount);
    }

    function getRevenueHistory(address business) external view returns (RevenueProof[] memory) {
        return revenueProofs[business];
    }

    function deactivateBusiness(address business) external onlyRole(DEFAULT_ADMIN_ROLE) {
        businesses[business].active = false;
    }

    function updateTreasury(address newTreasury) external onlyRole(DEFAULT_ADMIN_ROLE) {
        treasury = newTreasury;
    }
}
```

Notein
Scan the QR code to download Notein